When Valid Is Not Faithful: Verifier-Calibrated Search and Repair for Structured Generation

Pritam Mondal
Birla Institute of Technology and Science, Pilani

Abstract

Large language models increasingly translate natural language into structured artifacts such as programs, configurations, and planning problems. A central failure mode is that these artifacts can be valid in form while unfaithful in meaning. This gap between validity and faithfulness limits the reliability of LLM-based systems, especially agentic systems that translate user requests into plans or intermediate programs before acting. We study this valid-but-wrong problem in text-to-PDDL generation, where a generated planning problem can parse and even be solvable while formalizing the wrong task. We introduce VCSR, a verifier-calibrated search-and-repair framework for faithful structured generation. VCSR combines three ingredients: sample multiple candidates, select a promising parseable candidate with a learned semantic verifier, and repair the selected candidate with non-oracle feedback. On untouched final Planetarium seeds, full VCSR improves mean semantic equivalence at $K = 8$ from 0.368 for prompt-only generation to 0.772 over `blocksworld` and `gripper`, a gain of 40.4 percentage points. Verifier-only search improves to 0.420, making it a useful scaffold but not the final solution. The original $K = 8$ candidate-pool oracle is only 0.464; VCSR exceeds it because repair creates a new candidate outside the original pool. We also report negative evidence: planner/solvability search and further verifier training did not reliably solve semantic selection. The final system is a large net win, especially on `gripper`, but repair can hurt already-correct `blocksworld` candidates.

1 Introduction

Automated planning systems rely on formal models of states, actions, and goals. The Planning Domain Definition Language (PDDL) has long served as a standard representation for such models [Ghallab et al., 1998, Fox and Long, 2003]. If users could describe a task in natural language and reliably obtain faithful PDDL, classical planners would become substantially easier to deploy. Large language models make this goal tempting because they can produce structured, code-like artifacts from short descriptions. The danger is that a valid-looking artifact can still be the wrong formalization.

Planetarium makes this gap concrete. It evaluates text-to-PDDL translation by semantic equivalence rather than merely by parsing or plan existence [Zuo et al., 2024]. This distinction matters: a generated problem can parse and admit a plan while changing objects, initial facts, or goals. A planner may then solve the formal problem perfectly while solving the wrong user task. The same risk appears in agentic LLM systems: if an agent converts a request into an internal plan, tool call sequence, or executable specification, then a valid-looking intermediate artifact can still cause the agent to pursue the wrong objective [Yao et al., 2023, Schick et al., 2023].

A simple example. Imagine asking an assistant to turn "deliver the package to Purbita after Utpal signs for it" into a checklist. A checklist that says "Utpal signs after Purbita receives the

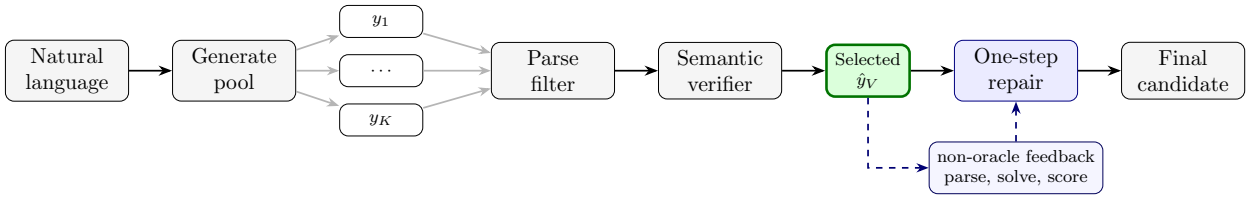

Figure 1: VCSR pipeline. The system samples a candidate pool, filters parseable PDDL, uses the verifier to select a promising candidate, and repairs that candidate using non-oracle feedback. The repair prompt never receives gold PDDL or an equivalence label.

package" may be perfectly well formatted, but it encodes the wrong task. Text-to-PDDL has the same problem in a stricter symbolic setting: a file can parse and even be solvable while reversing, dropping, or subtly changing the user's intended constraints. VCSR targets this broader problem of faithful formalization, with Planetarium providing a rigorous testbed.

Prior work has explored LLM-driven planning formalization, including NL2Plan [Gestrin et al., 2024], planning-domain generation [Oswald et al., 2024], and studies of the limits of LLMs as planning formalizers [Huang and Zhang, 2024]. These systems show that language models can be useful planning front ends, but they also reinforce the need for mechanisms that target semantic faithfulness rather than only syntactic validity or solvability.

We propose VCSR: Verifier-Calibrated Search and Repair. VCSR is built around the view that faithful text-to-PDDL generation requires a system, not only a larger prompt or a better verifier. The system should generate alternatives, score semantic plausibility, and repair promising but wrong formalizations. In short, faithful structured generation requires search and repair, not validation alone. Figure 1 summarizes the pipeline.

This paper makes four contributions. First, we present a search-and-repair framework for faithful structured generation, instantiated and evaluated on text-to-PDDL. Second, we use fixed-pool replay to separate candidate-pool randomness from verifier and policy changes. Third, we show on untouched final seeds that repair-augmented VCSR improves semantic equivalence over prompt-only generation, random parseable best-of-K, planner/solvability search, and verifier-only search. Fourth, we report negative results that clarify why the final gain comes from repair: additional verifier training and simple solvability-based search were not enough.

Code, configs, and selected experiment artifacts are available online.[1]

2 Related Work

Text-to-PDDL and planning formalization. Planetarium is the closest benchmark to our setting. It provides natural language descriptions paired with PDDL problems and evaluates generated problems using equivalence-based semantics [Zuo et al., 2024]. NL2Plan studies robust LLM-driven planning from minimal text descriptions [Gestrin et al., 2024]. Other recent work investigates LLM-generated planning domains [Oswald et al., 2024], generalized planning in PDDL domains with pretrained language models [Silver et al., 2024], and the degradation of LLM formalization under more natural descriptions [Huang and Zhang, 2024]. Our work differs in emphasizing semantic equivalence of generated problem files and in making repair a first-class system stage.

[1] `https://github.com/ultimatepritam/vcsr`

Best-of-K generation and test-time search. Sampling multiple outputs and selecting among them is a common way to improve language-model reasoning and generation. Self-consistency demonstrated the value of sampling diverse reasoning paths and aggregating outcomes [Wang et al., 2023]. More directly related to symbolic artifacts, recent work studies test-time scaling for generating symbolic world models [Yu et al., 2025]. VCSR follows the same broad test-time compute principle, but our final system does not only select from the original candidate pool; it repairs the verifier-selected candidate.

Repair and refinement. LLM refinement methods such as Self-Refine use feedback to iteratively improve model outputs [Madaan et al., 2023]. VCSR uses a narrower and more controlled repair stage: a single repair call is applied after verifier-guided selection, and the repair prompt receives no gold PDDL or equivalence label. This keeps the repair stage close to a deployable setting where the system can inspect its own candidate and lightweight diagnostics but does not know the ground truth.

Planning tools and weak correctness signals. Classical planners and validators remain important planning tools. Fast Downward is a widely used planning system [Helmert, 2006], and VAL supports plan validation for PDDL [Howey et al., 2004]. In this paper, however, solvability and validation are diagnostic signals rather than final correctness metrics. The primary metric is semantic equivalence to the intended task.

Calibration and selective prediction. During development we studied verifier calibration and risk-coverage, motivated by the miscalibration of neural confidence scores [Guo et al., 2017] and selective prediction work on risk-coverage tradeoffs [Geifman and El-Yaniv, 2017]. The final paper-facing mechanism is not abstention, however. We use the frozen verifier as a search scaffold and repair as the decisive system-level intervention.

3 Method

3.1 Task

We study text-to-PDDL problem generation in fixed in-domain Planetarium tasks. Each example consists of a natural-language description and a gold PDDL problem. A generated candidate is correct if it is semantically equivalent to the gold task under Planetarium's equivalence evaluator [Zuo et al., 2024, Bats Research, 2024b,a]. We evaluate on the `test` split and restrict the final claim to `blocksworld` and `gripper`. We make no final claim for `floor-tile`.

3.2 Problem Formalization

Let x denote a natural-language task and $y^\star$ denote its gold PDDL problem. A generator produces a candidate pool

$$C_K(x) = \{y_1, \ldots, y_K\}$$

under a test-time budget K. Planetarium provides a binary semantic equivalence evaluator

$$E(y, y^\star) \in \{0, 1\},$$

where $E(y, y^\star) = 1$ means that candidate y represents the same planning problem as the gold task. We also define the parseable subset of the candidate pool as

$$\mathcal{P}_K(x) = \{y \in C_K(x) : \text{parse}(y) = 1\}.$$

The verifier assigns each natural-language/candidate pair a score

$$s_\theta(x, y) \in [0, 1],$$

intended to approximate semantic faithfulness. The verifier-ranked selector is

$$\hat{y}_V(x) = \arg\max_{y \in \mathcal{P}_K(x)} s_\theta(x, y),$$

when at least one parseable candidate is available. For analysis only, we define the oracle best-of-K diagnostic

$$O_K(x) = \max_{y \in C_K(x)} E(y, y^\star).$$

This quantity is useful for measuring how much useful signal exists in the generated pool, but it is not a policy the system can run at test time: it requires gold equivalence labels after generation.

The deployable repair step instead starts from the verifier-selected candidate and applies a one-step transformation

$$r = R(x, \hat{y}_V(x), f(\hat{y}_V(x)))\,,$$

where $f(\hat{y}_V(x))$ is non-oracle feedback such as parse status, solvability status, and verifier score. The final repair-augmented policy at the main operating point $K = 8$ outputs this repaired candidate r; the non-repair baselines output their policy-specific selected candidate from $C_K(x)$. Because r need not belong to $C_K(x)$, repair-augmented VCSR can exceed the candidate-pool oracle $O_K(x)$: repair is allowed to create a new candidate, not merely choose among the original samples.

3.3 Candidate Generation

Given an input description x, the generator produces K candidate PDDL problems $C_K(x) = \{y_1, \ldots, y_K\}$ using stochastic decoding. Our main experiments use $K \in \{1, 4, 8\}$, with $K = 8$ as the primary operating point. Candidate outputs are parsed and normalized by the project evaluation stack. The $K = 1$ condition is the prompt-only baseline: the model is asked once, and the first candidate is evaluated directly.

3.4 Semantic Verifier

The verifier is a DeBERTaV3-base cross-encoder [He et al., 2021]. It scores pairs (x, y), where x is the natural-language task and y is a candidate PDDL problem, producing $s_\theta(x, y)$. The verifier is trained as a binary classifier over $(x, y, E(y, y^\star))$ examples derived from Planetarium equivalence: parseable equivalent candidates are positive examples, while parseable non-equivalent candidates and targeted perturbations are negative examples. The final paper-facing verifier is the round-4 checkpoint recorded in `results/verifier/best_current/selection.yaml`. Later verifier training experiments are treated as diagnostics and are not promoted.

3.5 Selection Policies

We evaluate four policies. `greedy_first` selects the first generated candidate. This is the prompt-only policy at $K = 1$ and a simple baseline at larger K. `random_parseable` selects uniformly from parseable candidates in $\mathcal{P}_K(x)$. `verifier_ranked` selects $\hat{y}_V(x)$, the parseable candidate with the highest frozen-verifier score. `verifier_ranked_repair` first selects the verifier-ranked candidate at $K = 8$ and then applies one repair call to produce r. None of these policies uses the gold PDDL or equivalence label at selection time.

3.6 One-Step Domain-Aware Repair

The final VCSR policy applies one repair call to the verifier-selected $K = 8$ candidate. The repair prompt receives the natural-language task, the selected candidate PDDL, and lightweight non-oracle feedback. It never receives the gold PDDL, the equivalence label, or an oracle-best candidate. For `blocksworld`, we use the generic repair prompt. For `gripper`, we use a Planetarium-specific repair prompt that encourages the object and predicate conventions needed by the benchmark. This domain-aware repair policy is the only promoted repair variant. A later score-guarded repair variant is reported as a negative follow-up because it did not reduce blocksworld hurts.

4 Experimental Setup

Dataset and final gate. The final evaluation uses Planetarium `test` rows from `blocksworld` and `gripper`. The primary final gate uses untouched seeds 51–55, 50 rows per seed, for 250 total rows. These seeds were not used for prompt tuning, verifier checkpoint selection, repair-prompt tuning, or guard selection. We evaluate $K \in \{1, 4, 8\}$ and use the frozen round-4 verifier from `results/verifier/best_current/selection.yaml`.

Metrics. The primary metric is semantic equivalence rate. We also report parse rate, equivalence given parse, domain and style slices, candidate-pool oracle diagnostics, and repair helped/hurt/tied counts. "Helped" means the original verifier-selected candidate was non-equivalent and the repaired candidate was equivalent. "Hurt" means the original was equivalent and repair made it non-equivalent. "Tied" includes both-success and both-fail cases.

Oracle best-of-K diagnostic. Oracle best-of-K, $O_K(x)$, is an offline candidate-pool diagnostic, not a deployable policy. It uses ground-truth equivalence labels after generation to ask whether at least one candidate in the original pool $C_K(x)$ is semantically equivalent. It is therefore an upper bound for policies that must choose among the original candidates. It is not an upper bound for repair-augmented VCSR, because repair creates a new candidate outside the original pool.

Replay and reproducibility. During development, we used cached candidate pools to replay alternative verifiers and policies on identical candidates. This avoided confounding policy changes with generator randomness and was used to reject additional verifier training rounds and planner/solvability-only search policies. The final evidence is stored under `results/vcsr/final_repair_gate_round4`; derived paper tables are stored under `results/paper/final_vcsr`. The project repository is available at `https://github.com/ultimatepritam/vcsr`.

K	Policy	Parse	Equiv.	Equiv./Parse
1	Prompt-only / first candidate	0.924	0.368	0.399
4	Greedy first	0.924	0.368	0.399
4	Random parseable	0.996	0.376	0.378
4	Verifier-ranked	0.996	0.392	0.394
8	Greedy first	0.924	0.368	0.399
8	Random parseable	1.000	0.376	0.376
8	Verifier-ranked	1.000	0.420	0.420
8	VCSR repair-augmented	1.000	**0.772**	**0.772**

Table 1: Final fresh-seed aggregate metrics on seeds 51–55. Semantic equivalence is the primary metric. VCSR repair is reported only at the main $K = 8$ operating point where repair is actually applied.

K	Avg. parseable candidates	Avg. equivalent candidates	Oracle best-of-K
1	0.924	0.368	0.368
4	3.812	1.464	0.428
8	7.564	2.888	0.464

Table 2: Candidate-pool diagnostics on final seeds 51–55. Oracle best-of-K, $O_K(x)$, uses gold equivalence labels and is only an offline upper bound for selecting among the original generated candidates. Repair can exceed this diagnostic because it creates a new candidate.

5 Results

5.1 Main Final Gate

Table 1 shows the final aggregate result over untouched seeds 51–55. The primary operating point is $K = 8$. Prompt-only generation reaches 0.368 semantic equivalence. Full repair-augmented VCSR reaches 0.772, which means 77.2% of rows are semantically correct. The absolute improvement over prompt-only is +0.404, or +40.4 percentage points. Verifier-only search reaches 0.420, confirming that the verifier has useful signal, but the large jump comes from repair.

5.2 Candidate-Pool Oracle

Table 2 reports candidate-pool diagnostics. At $K = 8$, the original generated pool $C_8(x)$ contains at least one semantically equivalent candidate, $O_8(x) = 1$, for only 46.4% of rows. This explains why verifier-only search has limited headroom: even a perfect selector over the original pool could not solve rows where no candidate is equivalent. Repair-augmented VCSR reaches 77.2% because it edits $\hat{y}_V(x)$ and can create a new equivalent PDDL problem. Repair exceeding oracle best-of-K is therefore expected: repair adds candidate quality rather than merely selecting better from the same pool.

5.3 Per-Seed Stability

Table 3 shows that repair improves every final seed. The smallest gain is +0.28 and the largest gain is +0.42. Repair parse rate is high across seeds, with a mean of 0.984.

Seed	Verifier $K = 8$	Repair $K = 8$	Delta	Helped	Hurt	Repair parse
51	0.360	0.780	+0.420	24	3	1.000
52	0.420	0.780	+0.360	21	3	0.980
53	0.520	0.800	+0.280	18	4	0.960
54	0.340	0.760	+0.420	22	1	0.980
55	0.460	0.740	+0.280	19	5	1.000

Table 3: Per-seed final gate outcomes. Repair improves semantic equivalence on all five untouched final seeds.

Slice	Policy	Equiv. count	Equiv. rate	Parse rate
domain=blocksworld	Greedy first	92/126	0.730	0.929
domain=blocksworld	Verifier-ranked	103/126	0.817	1.000
domain=blocksworld	VCSR repair-augmented	95/126	0.754	1.000
domain=gripper	Greedy first	0/124	0.000	0.919
domain=gripper	Verifier-ranked	2/124	0.016	1.000
domain=gripper	VCSR repair-augmented	98/124	0.790	1.000
style=abstract/abstract	Greedy first	28/126	0.222	0.849
style=abstract/abstract	Verifier-ranked	40/126	0.317	1.000
style=abstract/abstract	VCSR repair-augmented	84/126	0.667	1.000
style=explicit/explicit	Greedy first	64/124	0.516	1.000
style=explicit/explicit	Verifier-ranked	65/124	0.524	1.000
style=explicit/explicit	VCSR repair-augmented	109/124	0.879	1.000

Table 4: $K = 8$ domain and style slices. Repair strongly improves gripper and both style groups, but can hurt already-correct blocksworld selections.

5.4 Domain and Style Breakdown

Table 4 highlights the main caveat. Repair is transformative on `gripper`: verifier-ranked search solves only 2 of 124 rows, while repair solves 98 of 124. In `blocksworld`, verifier-ranked search is already stronger, and unconditional repair hurts some already-correct candidates: blocksworld drops from 103/126 to 95/126. Overall, the gripper gains dominate, but the domain asymmetry is important.

5.5 Repair Outcomes

Across the final gate, repair helped 104 rows, hurt 16 rows, and tied 130 rows. The hurt rows are concentrated in blocksworld, where the verifier-selected candidate is often already correct. Figure 2 summarizes the outcome counts.

6 Ablations and Analysis

Table 5 summarizes the development evidence. The project began with verifier training, but the final result is a system-level discovery: verifier-only search helps, simple planner/search policies do not reliably improve semantic selection, and repair provides the decisive jump.

Repair outcome	Count
Helped	104
Hurt	16
Both success	89
Both fail	41

Figure 2: Final repair outcomes on seeds 51–55. Repair is a large net win but not uniformly harmless.

Ablation	Evidence source	Key result	Decision
Verifier-only search	Final seeds 51–55	Prompt-only 0.368 → verifier-ranked 0.420 at $K = 8$	Useful scaffold, not sufficient
Solvability/planner search	Cached search replay over six pools	Best solvability policy gave only +0.004 over verifier-ranked at $K = 8$; parse-solvable index fell to 0.418	No robust selection gain
Additional verifier training	Round-7 fixed-pool comparison	Round 7 gave +0.0067 at $K = 8$ but -0.0133 at $K = 4$ on fresh fixed pools	Keep round 4 frozen
Guarded repair	Fresh seeds 67–71	Reached 0.796 at $K = 8$ but did not reduce blocksworld hurts	Do not promote guard
Repair-augmented VCSR	Final seeds 51–55	Verifier-ranked 0.420 → repair 0.772 at $K = 8$	Promoted system

Table 5: Development ablations. Values are semantic equivalence rates unless otherwise noted. The table explains why the final system uses a frozen verifier plus repair rather than solvability filtering, further verifier training, or a score guard.

Verifier-only search is useful but insufficient. Verifier-ranked selection improves over prompt-only generation at $K = 8$, raising semantic equivalence from 0.368 to 0.420 on the final gate. This confirms that the verifier has useful ranking signal. However, the gain is small compared with repair, so the verifier is best viewed as a scaffold for identifying a candidate to repair rather than as the full solution.

Planner and solvability signals are weak semantic selectors. Planner and solvability policies did not pass the cached replay gate. This is consistent with the motivation from Planetarium: solvability is not semantic faithfulness. A solvable PDDL problem can still encode the wrong initial state or goal.

Additional verifier training did not solve selection. We trained several verifier successors, including ranking-aligned and focused pointwise variants. Some improved offline metrics or cached replay, but none beat the frozen round-4 verifier sufficiently to become the paper-facing selector. The remaining bottleneck was not solved by simply training the same verifier harder.

Guarded repair did not fix blocksworld hurts. A simple verifier-score guard was tested on fresh seeds 67–71. It replicated the large repair gain but did not reduce blocksworld hurts relative to unconditional repair, so it is not promoted. This suggests that future repair acceptance checks should use stronger structural signals rather than only the same verifier score.

7 Broader Applicability

Although our experiments use Planetarium text-to-PDDL as the testbed, the failure mode is not specific to planning. Many LLM applications ask a model to produce a structured artifact whose syntax can be checked more easily than its meaning: code that compiles but implements the wrong behavior, configuration files that validate but encode the wrong deployment, database queries that run but answer the wrong question, or domain-specific language programs that satisfy a grammar while violating user intent. The issue is especially relevant for agentic workflows, where structured plans or tool-call programs may be executed with real downstream consequences [Yao et al., 2023, Schick et al., 2023]. In these settings, validity checks are necessary but not sufficient. The central design pattern behind VCSR is to combine three ingredients: sample multiple candidates, use a task-specific semantic signal to choose a promising artifact, and then repair that artifact with non-oracle feedback. Our results do not prove that the same implementation will transfer unchanged to every structured-generation domain, but they support the broader claim that faithful structured generation often requires search and repair, not only syntactic validation.

8 Limitations

The final claim is intentionally scoped. First, we evaluate only in-domain Planetarium `blocksworld` and `gripper`; we make no final claim for `floor-tile` or arbitrary new planning domains. Second, the repair stage is domain-aware. This is appropriate for our in-domain system, but it may limit immediate transfer to new domains. Third, repair is not uniformly harmless: the final system is a large net win, but blocksworld shows that repairing an already-correct candidate can introduce errors. Fourth, the verifier remains imperfect and text-only; richer semantic validators, graph-based verifiers, or structural repair acceptance checks may reduce the remaining hurts. Finally, calibration and abstention were explored during development but are not the final paper-facing mechanism.

9 Conclusion

VCSR addresses a central failure mode in text-to-PDDL generation: generated planning problems can be formally plausible yet semantically wrong. Our results show that verifier-ranked best-of-K search alone is helpful but insufficient. The decisive improvement comes from adding one repair step to the verifier-selected candidate. On untouched final seeds, repair-augmented VCSR raises semantic equivalence from 36.8% for prompt-only generation to 77.2% at the main $K = 8$ operating point, while maintaining high parseability. The lesson is practical: if the goal is faithful planning formalization, solvability and one-shot prompting are not enough; semantic search and targeted repair are complementary ingredients. While our experiments are scoped to Planetarium text-to-PDDL tasks, the underlying pattern is broader: systems that translate natural language into structured executable artifacts need mechanisms for checking and repairing meaning, not only syntax.

Model	Prompt $K = 1$	VCSR repair $K = 8$	Delta vs prompt	Random $K = 8$	Verifier $K = 8$
Claude Haiku 4.5	0.400	0.900	+0.500	0.300	0.467
Claude Sonnet 4.5	0.500	0.933	+0.433	0.467	0.533
Claude Opus 4.6	0.367	0.900	+0.533	0.433	0.433

Table 6: Small post-freeze Claude-family robustness benchmark. These results support the view that VCSR is a useful wrapper, but the main paper evidence remains the untouched final seed 51–55 gate.

A Claude-Family Robustness Benchmark

After freezing the main result, we ran a small Claude-family robustness benchmark on new seeds 72–74 with 10 rows per seed and model. This benchmark is supporting evidence, not a replacement for the final seed 51–55 result. Table 6 compares prompt-only generation against full VCSR for Claude Haiku 4.5, Claude Sonnet 4.5, and Claude Opus 4.6. All three improve substantially under VCSR. With prompt-only Opus, about 36.7% of rows were semantically correct; with full VCSR around Opus, 90.0% were semantically correct, a +53.3 percentage-point gain on this small benchmark.

B Development Evidence Summary

Round 4 remains the frozen verifier because it was the strongest checkpoint under the replay and fresh held-out gates. Pairwise round 5, conservative ranking round 6, and focused pointwise round 7 were useful diagnostics but were not promoted. This history is important because it shows that the final gain is not simply the result of training the verifier longer; the decisive improvement comes from adding repair to the verifier-selected candidate.

References

Bats Research. Planetarium code repository. `https://github.com/BatsResearch/planetarium`, 2024a. GitHub repository.

Bats Research. Planetarium dataset. `https://huggingface.co/datasets/BatsResearch/planetarium`, 2024b. Hugging Face dataset card.

Maria Fox and Derek Long. Pddl2.1: An extension to pddl for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003. doi: 10.1613/jair.1129. URL `https://www.cs.cmu.edu/afs/cs/project/jair/pub/volume20/fox03a-html/JAIRpddl.html`.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, volume 30, pages 4878–4887, 2017. URL `https://papers.neurips.cc/paper/7073-selective-classification-for-deep-neural-networks`.

Elliot Gestrin, Marco Kuhlmann, and Jendrik Seipp. Nl2plan: Robust llm-driven planning from minimal text descriptions. *arXiv preprint arXiv:2405.04215*, 2024. URL `https://arxiv.org/abs/2405.04215`.

Malik Ghallab, Adele Howe, Craig Knoblock, Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. Pddl: The planning domain definition language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control, 1998. URL https://www.isi.edu/results/publications/62624/pddl-the-planning-domain-definition-language/.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017. URL https://proceedings.mlr.press/v70/guo17a.html.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing. *arXiv preprint arXiv:2111.09543*, 2021. URL https://arxiv.org/abs/2111.09543.

Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006. doi: 10.1613/jair.1705. URL https://jair.org/index.php/jair/article/view/10457.

Richard Howey, Derek Long, and Maria Fox. Val: Automatic plan validation, continuous effects and mixed initiative planning using pddl. In *16th IEEE International Conference on Tools with Artificial Intelligence*, pages 294–301, 2004. doi: 10.1109/ICTAI.2004.120.

Cassie Huang and Li Zhang. On the limit of language models as planning formalizers. *arXiv preprint arXiv:2412.09879*, 2024. URL https://arxiv.org/abs/2412.09879.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023. URL https://arxiv.org/abs/2303.17651.

James Oswald, Kavitha Srinivas, Harsha Kokel, Junkyu Lee, Michael Katz, and Shirin Sohrabi. Large language models as planning domain generators. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 423–431, 2024. doi: 10.1609/ICAPS.V34I1.31502. URL https://research.ibm.com/publications/large-language-models-as-planning-domain-generators.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2302.04761.

Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. Generalized planning in pddl domains with pretrained large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 20256–20264, 2024. URL https://research.ibm.com/publications/generalized-planning-in-pddl-domains-with-pretrained-large-language-models--1.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in lan-

guage models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=1PL1NIMMrw.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Zhouliang Yu, Yuhuan Yuan, Tim Z. Xiao, Fuxiang Frank Xia, Jie Fu, Ge Zhang, Ge Lin, and Weiyang Liu. Generating symbolic world models via test-time scaling of large language models. *arXiv preprint arXiv:2502.04728*, 2025. URL https://arxiv.org/abs/2502.04728.

Max Zuo, Francisco Piedrahita Velez, Xiaochen Li, Michael L. Littman, and Stephen H. Bach. Planetarium: A rigorous benchmark for translating text to structured planning languages. *arXiv preprint arXiv:2407.03321*, 2024. URL https://arxiv.org/abs/2407.03321.